

Instructions for Assignment Submission

1. File Naming and Submission:
 - a. Create a ZIP file containing all your solution files.
 - b. Name the ZIP file as `assignment03_<ERP>.zip`, where `<ERP>` is your ERP.
 - c. Ensure that the ZIP file contains:
 - i. C++ source files
 - ii. No unnecessary files (such as executables, temporary files, or editor configs)
 - d. Do not include `TEXT(.txt)` files.
2. Code Neatness:
 - a. Ensure your code is well-formatted and easy to read. Use proper indentation and meaningful variable names.
 - b. Include appropriate comments to explain the logic where necessary (especially for functions and complex sections of the code).
 - c. Add clear explanations where required (or specified) to make the logic of your program easy to follow.
3. Code Compilation and Functionality:
 - a. Ensure your code compiles without errors or warnings. You should be able to compile and run your code on any system with a standard C++ compiler (e.g., GCC, Clang, MSVC).
 - b. Double-check that the functionality of the program meets the requirements outlined in the task.
4. Additional Instructions:
 - a. Do not use any libraries or features that have not been covered in the course material, unless specifically instructed.
 - b. Make sure your code adheres to the principles of good software development (e.g., minimal repetition, and no hardcoding).
 - c. If you encounter any issues or require clarifications, reach out before the submission deadline.
5. Deadline:
 - a. Ensure that you submit the assignment on time. Late submissions will not be accepted.
 - b. Double-check that the zip file contains all required files before submitting.
6. Plagiarism Policy:
 - a. The work submitted must be entirely your own. Any code or content copied from others (including online sources, classmates or AI) will result in an automatic zero for the task.

Note: You may optionally submit your assignment via GitHub. Well-documented GitHub submissions with a clear and detailed README will receive a bonus.

1 Sorting a String

Write a function that alphabetically sorts the characters in a given string. The function should take an additional argument that specifies how many characters (from the beginning) to sort. By default, the function should sort the entire string.

If a valid number N is provided, then only the first N characters should be sorted, while the rest of the string should remain unchanged. You may use any simple sorting algorithm of your choice (such as insertion sort or selection sort), but do not use `std::sort` or any built-in sorting functions.

The function should return the sorted string.

1.1 Function prototype:

```
std::string sort_string(const std::string& str, int n = -1);
```

After implementing this function, think about how the code would need to change if we wanted to make it sort the string in place (modifying the original). Then, implement a second version of the function:

- `sort_string` — returns a new sorted string.
- `sort_string_inplace` — modifies the original string.

```
Enter a string: This is Introduction to Programming
Enter number of characters to sort (negative for all): 10
Sorted string (new):  hiiInssTtroductio n to Programming
Sorted string (in place):  hiiInssTtroductio n to Programming
```

Use clear, reusable logic and avoid redundant code between the two functions. Make the sorting case-insensitive (i.e., 'A' and 'a' are considered equal in order).

2 File Name Validator

Write a function that verifies whether a submitted set of files follows the required naming conventions.

2.1 Background:

Each student submission must be compressed into a single `.zip` file named using the following format:

`lab<lab_number>_<erp>.zip`

Inside that archive, each source file must follow the pattern:

`task<task_number>.cpp`

2.2 Function requirements:

The function should take three main inputs: an integer representing the lab number, an integer for the ERP number, and a `std::vector<std::string>` that holds the file names to be checked. The first element of this vector (index 0) represents the name of the `.zip` file, while the remaining elements correspond to the names of the files contained within it. The function must return a boolean value — `true` if all names conform to the required format, and `false` otherwise. Additionally, it should accept an optional `bool verbose = false` parameter; when enabled, the function prints a validation message for each file, indicating whether it passed or failed the checks.

Validation Rules:

- The zip file name must:
 - Begin with `lab`.
 - Immediately follow with the correct lab number.
 - Include an underscore `_`.
 - Include the ERP number.
 - End with `.zip`.
- Each file inside the zip (starting from index 1) must:
 - Be named in the format `task<i>.cpp`.
 - Contain no spaces or special characters (only letters, digits, underscores, and periods are allowed).

You may assume there are no leading zeroes in task or lab numbers. For example, `task1.cpp` and `lab3_12345.zip` are valid, while `task01.cpp` and `lab03_12345.zip` need not be handled.

Expected function signature:

```
bool valid_submission(int labNumber, int erp, const std::vector<std::string>& files, bool verbose = false);
```

In `main()`:

- Write a small test program that calls `validate_submission()` with several test cases:
 - One valid submission.
 - One with an incorrect zip file name.
 - One with an invalid task file name.
 - One with special characters or spaces.
- Print whether each case passed or failed.

```

Verbose Mode:
Validating submission: lab8_12345.zip
- lab8_12345.zip           pass
- task1.cpp                pass
- task2.cpp                pass
- task3.cpp                pass
Summary for lab8_12345.zip: 4/4 passed

Validating submission: lab08_12345.zip
- lab08_12345.zip         fail
- task1.cpp               pass
- task2.cpp               pass
Summary for lab08_12345.zip: 2/3 passed

Validating submission: lab8_12345.zip
- lab8_12345.zip           pass
- task01.cpp               fail
- task2.cpp                pass
Summary for lab8_12345.zip: 2/3 passed

Validating submission: lab8_12345.zip
- lab8_12345.zip           pass
- task1!.cpp               fail
- task2.cpp                pass
Summary for lab8_12345.zip: 2/3 passed

Non-Verbose Mode:
- Valid Submission         pass
- Invalid Zip Name         fail
- Invalid Task Name        fail
- Special Character in Task Name fail

```

3 Command-Line Calculator with History

Implement a simple command-line calculator that supports basic arithmetic operations: addition (+), subtraction (-), multiplication (*), and division (/).

The program should repeatedly:

- Read a user command.
- Parse and evaluate expressions of the form:

`<operator> <num1> <num2>`

- Display the result or an appropriate error message.

Additional commands:

- `history` — show the history of all previous operations.
- `help` — display a short help menu explaining available commands.
- `quit` — exit the program.

Use `std::vector<int [3]>` to store history entries, where each record represents:

`{first_num, op, second_num, result}`

To print the operator, cast it back from its integer form:

`static_cast<char>(entry[1])`

Example interaction:

```
Simple Command-Line Calculator
Type 'help' for a list of commands.

> help
Usage:
    <operator> <num1> <num2>

Description:
    Perform basic arithmetic operations or run utility commands.

Operators:
    +, -, *, /          perform arithmetic between two numbers
                        Example: + 5 3    ->  8

Commands:
    history             show history of all previous operations
    help               display this help message
    quit              exit the program

> + 2 3
5

> / 2 0
Error: Division by zero.

> history
2 + 3 = 5

> quit
```

Use a `switch` statement to handle arithmetic operations, and structure the program into clear, reusable functions.

4 Recursive Maze Solver

Write a recursive function that finds a path through a maze represented as a 2D character array.

4.1 Background

A maze is represented as a grid where:

- '.' represents an open path
- '#' represents a wall
- 'S' represents the start position
- 'E' represents the exit

Your task is to implement a recursive pathfinding algorithm that explores the maze to find a route from the start to the exit.

4.2 Function Requirements

Implement the following recursive function:

```
bool solve_maze(char maze[][MAX_COLS], int rows, int cols,  
                int x, int y, std::vector<std::pair<int,int>>& path);
```

Where:

- **maze** is the 2D character array representing the maze
- **rows** and **cols** are the dimensions of the maze
- **x** and **y** are the current position coordinates
- **path** stores the solution path as coordinate pairs

The function should:

- Use recursion to explore all four directions: up, down, left, right
- Mark visited cells to avoid infinite loops
- Store the solution path in the **path** vector
- Return **true** if a path to the exit is found, **false** otherwise

4.3 Algorithm Rules

1. You can only move to adjacent cells (not diagonally).
2. You cannot move through walls ('#') or revisit already visited cells.
3. When exploring a cell, mark it as visited using '*'.
4. If the recursion reaches a dead end, backtrack by restoring the cell to '.'.
5. The recursion should try all four directions before backtracking.

4.4 Recursive Structure

Base Cases:

- If current position is the exit 'E', return **true** (solution found)
- If current position is out of bounds, return **false**
- If current position is a wall '#' or already visited '*', return **false**

Recursive Case:

1. Mark the current cell as visited ('*')
2. Add current position to the path vector
3. Try moving in all four directions recursively
4. If any direction leads to the exit, return **true**
5. If no direction works, backtrack:

- Remove current position from path vector
- Restore current cell to '.' (unless it's the start)
- Return `false`

4.5 Sample Input/Output

```
Enter maze dimensions (rows cols): 5 7
Enter maze row by row (# for walls, . for path, S for start, E for exit):
Row 1: #####
Row 2: #S...#
Row 3: #.###.
Row 4: #...#E#
Row 5: #####

Solving maze...

Solution found!
Path length: 9

Final maze with solution path:
#####
#S*****
#*#####
#***#E#
#####

Path coordinates:
(1,1) -> (1,2) -> (1,3) -> (1,4) -> (1,5) -> (2,5) -> (3,5) -> (3,4) -> (3,3) -> (3,2) ->
(3,1)
```

4.6 In `main()`

Your main function should:

- Read maze dimensions from the user
- Read the maze contents row by row
- Validate that the maze contains exactly one 'S' and one 'E'
- Find the starting position coordinates
- Call the recursive `solve_maze` function
- If a solution exists:
 - Display “Solution found!”
 - Print the path length
 - Display the maze with the solution path marked
 - Print the path coordinates
- If no solution exists:
 - Display “No solution exists”
 - Show the maze with explored paths

4.7 Additional Requirements

- Define `MAX_COLS` as a constant (e.g., 50) for the maze width limit
- Maximum maze size: 50×50
- Add comments explaining the recursive logic
- Handle edge cases: no start/exit, multiple starts/exits, empty maze

Assignment #03

29th November, 2025

- The start position 'S' should remain marked as 'S' in the final output, not '*'

4.8 Hints

- Start by finding the coordinates of 'S' by scanning the maze
- Use helper functions to check if a position is valid:

```
bool is_valid(int x, int y, int rows, int cols);
```

- The four directions can be represented using direction arrays:

```
int dx[] = {-1, 1, 0, 0}; // up, down, left, right  
int dy[] = {0, 0, -1, 1};
```

- Remember to preserve the 'S' marker when backtracking from the start position
- Test with mazes that have no solution to verify backtracking works correctly
- Consider printing the maze after each recursive step (with a small delay) to visualize the algorithm

4.9 Example: No Solution Case

```
Enter maze dimensions (rows cols): 5 5  
Enter maze row by row:  
Row 1: #####  
Row 2: #S..#  
Row 3: ###.#  
Row 4: #.#E#  
Row 5: #####  
  
Solving maze...  
  
No solution exists.  
  
Explored maze:  
#####  
#S***#  
#####  
#.#E#  
#####  
  
Dead ends encountered: 3
```